

QR Payload Decode (Base64 → Latin-1) for MMMOCL

Java (Android) and Node.js reference snippets

Overview

For MMMOCL QR generation, the payload is provided as a Base64-encoded string. After decoding, the resulting bytes should be interpreted using Latin-1 (ISO-8859-1) so the byte-to-character mapping is preserved 1:1. The Latin-1 string is then used as the QR content.

Java (Android)

Steps:

- Read the Base64 string.
- Decode to bytes.
- Convert bytes to a Latin-1 string (ISO-8859-1).
- Use the Latin-1 string as QR content.

Sample code:

```
// Example Base64 string
String base64String = "rN2hRC4KzOJs7B8PuQvBqInkG1PCOtYecTW+hVqwib0=";

// Decode the Base64 string (Android)
byte[] decodedBytes = Base64.decode(base64String, Base64.DEFAULT);

// (Optional) Convert the byte array to a default charset string
String decodedString = new String(decodedBytes);
System.out.println("Decoded string: " + decodedString);

try {
    // Convert the byte array to a string with Latin-1 encoding
    String decodedString_latin = new String(decodedBytes, "ISO-8859-1");

    // Use decodedString_latin to create QR code
    System.out.println("Decoded string latin: " + decodedString_latin);
} catch (UnsupportedEncodingException e) {
    e.printStackTrace();
}
```

Node.js

Equivalent decoding logic in Node.js:

```
const data = qr_data_coming_from_onc;    // Base64 string
const buffer = Buffer.from(data, 'base64');
const latin1String = buffer.toString('latin1');

// Use latin1String as QR content
```

Notes

- Latin-1 (ISO-8859-1) preserves the exact byte values when converting bytes to a string (0–255).
- Avoid UTF-8 decoding for this use-case unless the upstream bytes are known to be valid UTF-8 text.
- Generate the QR code using the Latin-1 string as-is (no additional transformations).